

University of Wah

Department of Computer Science



Submit to:

Ms. Hassan Shafaqat

Submit by:

Muhammad Talha

Course:

Information and Security_Lab - CS-251L

Reg No:

UW-24-CS-BS-101

Section:

BS-CS-3rd-B

Open Ended Lab

Question # 01

```
# 1. Parse & Normalize Logs
# -----
def parse_logs(raw_logs):
    logs = []
    for line in raw_logs:
        timestamp, emp_id, access_point, result = line.strip().split(
            ", ")
        logs.append({
            "timestamp": datetime.strptime(timestamp, "%Y-%m-%d %H
            :%M:%S"),
            "employee_id": emp_id,
            "access_point": access_point,
            "access_result": result
        })
    return logs

# 2. Detect Logically Impossible Patterns
# -----
def detect_impossible_access(logs):
    seen = {}
    suspicious = set()

    for log in logs:
        key = (log["employee_id"], log["timestamp"])
        if key in seen and seen[key] != log["access_point"]:
            suspicious.add(log["employee_id"])
        seen[key] = log["access_point"]

    return suspicious
```

```

def detect_abnormal_frequency(logs):
    access_count = Counter(log["employee_id"] for log in logs)
    counts = list(access_count.values())

    if len(counts) < 2:
        return set(), access_count

    mean = statistics.mean(counts)
    std = statistics.stdev(counts)

    abnormal = {
        emp for emp, cnt in access_count.items()
        if cnt > mean + 2 * std
    }

    return abnormal, access_count

```

```

def compute_risk_scores(logs):
    risk_scores = Counter()
    impossible = detect_impossible_access(logs)
    abnormal, _ = detect_abnormal_frequency(logs)
    # Add risk points
    for emp in impossible:
        risk_scores[emp] += 5
    for emp in abnormal:
        risk_scores[emp] += 3
    for log in logs:
        if log["access_result"] == "DENIED":
            risk_scores[log["employee_id"]] += 1

    return risk_scores

```

```

def rank_employees(risk_scores):
    return sorted(risk_scores.items(), key=lambda x: x[1], reverse=True)

```

```

if __name__ == "__main__":

    raw_logs = [
        "2026-01-25 09:00:00, E101, Gate_A, GRANTED",
        "2026-01-25 09:00:00, E101, Gate_B, GRANTED",
        "2026-01-25 22:10:00, E102, Gate_A, DENIED",
        "2026-01-25 22:11:00, E102, Gate_A, DENIED",
        "2026-01-25 10:30:00, E103, Gate_C, GRANTED",
        "2026-01-25 10:31:00, E103, Gate_C, GRANTED",
        "2026-01-25 10:32:00, E103, Gate_C, GRANTED"
    ]

    logs = parse_logs(raw_logs)
    risk_scores = compute_risk_scores(logs)
    ranked_employees = rank_employees(risk_scores)

    print("\n Suspicious Employees (Ranked):\n")
    for emp, score in ranked_employees:
        print(f"Employee ID: {emp} | Risk Score: {score}")

```

Output

```

Suspicious Employees (Ranked):

Employee ID: E101 | Risk Score: 5
Employee ID: E102 | Risk Score: 2

```

Question # 02

```

from datetime import datetime
from collections import defaultdict

DANGEROUS_PERMISSION_RULES = [
    {"CAMERA", "INTERNET"},
    {"MIC", "INTERNET"},
    {"STORAGE", "INTERNET"}
]

VPN_DATA_THRESHOLD = 5.0 # MB

```

```

def parse_logs(raw_logs):
    for line in raw_logs:
        ts, dev, perms, data, vpn = line.split(", ")
        yield {
            "timestamp": datetime.strptime(ts, "%Y-%m-%d %H:%M:%S"),
            "device": dev,
            "permissions": set(perms.split("|")),
            "data": float(data),
            "vpn": vpn
        }

def analyze_logs(logs):
    findings = defaultdict(list)

    for log in logs:
        device = log["device"]

        for rule in DANGEROUS_PERMISSION_RULES:
            if rule.issubset(log["permissions"]):
                findings[device].append("Dangerous Permission
                    Combination")

        if log["vpn"] == "OFF" and log["data"] > VPN_DATA_THRESHOLD:
            findings[device].append("High Data Transfer Without VPN")

        if log["data"] > 10:
            findings[device].append("Unusual Background Data Upload")

    return findings

def calculate_risk(findings):
    risk_scores = {}

    for device, issues in findings.items():
        score = 0

```

```

for device, issues in findings.items():
    score = 0
    for issue in issues:
        if issue == "Dangerous Permission Combination":
            score += 3
        elif issue == "High Data Transfer Without VPN":
            score += 5
        elif issue == "Unusual Background Data Upload":
            score += 2
    risk_scores[device] = score

return risk_scores

```

```

def generate_report(risk_scores):
    return sorted(risk_scores.items(), key=lambda x: x[1], reverse=True)

```

```

if __name__ == "__main__":

```

```

    raw_logs = [
        "2026-01-25 10:00:00, D201, CAMERA|INTERNET, 2.5, ON",
        "2026-01-25 10:10:00, D201, CAMERA|INTERNET, 8.2, OFF",
        "2026-01-25 11:00:00, D202, STORAGE|INTERNET, 12.4, ON",
        "2026-01-25 12:00:00, D203, MIC, 0.3, ON"
    ]

```

```

logs = parse_logs(raw_logs)
findings = analyze_logs(logs)
risk_scores = calculate_risk(findings)
report = generate_report(risk_scores)

print("\n🔍 Device-Level Security Assessment Report (Rule-Based)\n")
for device, score in report:
    print(f"Device: {device} | Risk Score: {score}")

```

Output

```

🔍 Device-Level Security Assessment Report (Rule-Based)

Device: D201 | Risk Score: 11
Device: D202 | Risk Score: 5

```

Question # 03

```
from datetime import datetime
from collections import defaultdict

def parse_physical_logs(raw_logs):
    events = []
    for line in raw_logs:
        ts, person, location, action = line.split(", ")
        events.append({
            "time": datetime.strptime(ts, "%Y-%m-%d %H:%M:%S"),
            "type": "PHYSICAL",
            "person": person,
            "location": location,
            "action": action
        })
    return events

def parse_system_logs(raw_logs):
    events = []
    for line in raw_logs:
        ts, system, event = line.split(", ")
        events.append({
            "time": datetime.strptime(ts, "%Y-%m-%d %H:%M:%S"),
            "type": "CYBER",
            "system": system,
            "event": event
        })
    return events

def build_unified_timeline(physical, cyber):
    timeline = physical + cyber
    timeline.sort(key=lambda x: x["time"])
    return timeline

def reconstruct_security_timeline(timeline):
    security_state = "STRONG"
    active_personnel = set()
    suspicious_timeline = []

    for event in timeline:
```

```
for event in timeline:
```

```
    if event["type"] == "PHYSICAL":
        if event["action"] == "ENTER":
            active_personnel.add(event["person"])
        elif event["action"] == "EXIT":
            active_personnel.discard(event["person"])

    if event["type"] == "CYBER":
        if event["event"] == "ALARM_DISABLED":
            security_state = "WEAKENED"
        elif event["event"] == "ALARM_ENABLED":
            security_state = "STRONG"
```

```
        if event["event"] == "CONFIG_CHANGED" and security_state == "WEAKENED":
            suspicious_timeline.append({
                "time": event["time"],
                "issue": "Config Change During Weakened Security",
                "active_personnel": list(active_personnel)
            })
```

```
        if security_state == "WEAKENED" and active_personnel:
            suspicious_timeline.append({
                "time": event["time"],
                "issue": "Personnel Present During Weakened Security",
                "active_personnel": list(active_personnel)
            })
```

```
    return suspicious_timeline
```

```
def find_breach_window(suspicious_events):
```

```
    if not suspicious_events:
        return None
```

```
    start = suspicious_events[0]["time"]
    end = suspicious_events[-1]["time"]
```

```
    return start, end
```



```

if __name__ == "__main__":

    physical_logs = [
        "2026-01-25 22:00:00, EMP301, ServerRoom, ENTER",
        "2026-01-25 22:20:00, EMP301, ServerRoom, EXIT"
    ]

    system_logs = [
        "2026-01-25 21:55:00, SecuritySystem, ALARM_DISABLED",
        "2026-01-25 22:05:00, Server01, CONFIG_CHANGED",
        "2026-01-25 22:30:00, SecuritySystem, ALARM_ENABLED"
    ]

    timeline = build_unified_timeline(physical_events, cyber_events)
    suspicious_timeline = reconstruct_security_timeline(timeline)
    breach_window = find_breach_window(suspicious_timeline)

    print("\n Unified Suspicious Timeline:\n")
    for event in suspicious_timeline:
        print(
            f"{event['time']} | {event['issue']} | "
            f"Personnel: {event['active_personnel']}"
        )

    if breach_window:
        print("\nMost Probable Breach Window:")
        print(f"From {breach_window[0]} to {breach_window[1]}")

```

Output

```

Unified Suspicious Timeline:

2026-01-25 22:00:00 | Personnel Present During Weakened Security | Personnel
: ['EMP301']
2026-01-25 22:05:00 | Config Change During Weakened Security | Personnel:
['EMP301']
2026-01-25 22:05:00 | Personnel Present During Weakened Security | Personnel
: ['EMP301']

Most Probable Breach Window:
From 2026-01-25 22:00:00 to 2026-01-25 22:05:00

```